AI Project Report

Smart Study Planner

AI-Based Personalized Study Scheduling Agent

Submitted by: Amjad Ali Abro 023-23-0043
Sarfraz Ali Katper 023-23-0045
Supervisor: Dr. Muhammad Ismail
Semester: 6th Sukkur IBA University

Project and presentation video drive link

https://drive.google.com/file/d/1vQdLRa3fwokirtt4R-3i04_2iepkACHj/view?usp=sharing

ABSTRACT

Smart Study Planner is an AI-powered web application built using Python and Streamlit that helps university students create personalized, data-driven study schedules. The system uses two trained Machine Learning models — a Random Forest Regression model and a Random Forest Classification model — to predict the total study hours required for each subject and automatically assign a priority level (High, Medium, or Low) based on subject difficulty, chapters remaining, days until exam, daily free hours, and mid-term score.

The application has four pages: AI Planner (subject input and ML schedule generation), Dashboard (progress overview and weekly study plan), My Subjects (subject management and rescheduling), and Analytics (charts and data summary). It addresses the common problem of poor study time management by replacing manual guesswork with intelligent, ML-driven recommendations, and includes a professional UI, interactive Plotly charts, and CSV export.

TABLE OF CONTENTS

1. Introduction
2. Literature Review
3. System Design and Architecture
4. Implementation
5. Machine Learning Methodology
6. Conclusion and Future Work

1.INTRODUCTION

1.1 Background and Problem Statement

University students typically manage 5-8 subjects at once, each with different difficulty levels, syllabus lengths, and exam dates. Manually estimating how much time each subject needs, prioritizing correctly, and adapting when a study session is missed is difficult and error-prone. Existing tools (calendars, to-do apps) rely entirely on manual input and do not adapt or predict anything. Key problems:

- Students cannot accurately estimate required study hours.
- Priority is assigned manually, not based on objective urgency.
- No visual tracking of progress across subjects.
- No automatic rescheduling when sessions are missed.

1.2 Objectives

1. Build an AI-powered web app that generates personalized study schedules.
2. Train and integrate two ML models — Regression (study hours) and Classifier (priority).
3. Provide a real-time dashboard with progress, deadlines, and a weekly study chart.
4. Provide an analytics page with interactive charts.
5. Implement adaptive rescheduling for missed sessions.
6. Deliver a professional, modern UI.

1.3 Scope

The project covers the full ML pipeline (feature engineering, training, serialization), a four-page Streamlit app with custom CSS, interactive Plotly visualizations, subject management, and CSV export. Multi-user accounts, databases, and mobile apps are out of scope (future work).

2. LITERATURE REVIEW

Intelligent Tutoring Systems (ITS) have shown that personalized, adaptive instruction improves learning outcomes (VanLehn, 2011), tracing back to early systems like SCHOLAR (Carbonell, 1970). Educational Data Mining and Learning Analytics (Romero & Ventura, 2020) apply ML to understand and improve student outcomes, and Random Forest models have proven effective on tabular educational data (Baker & Inventado, 2014; Breiman, 2001).

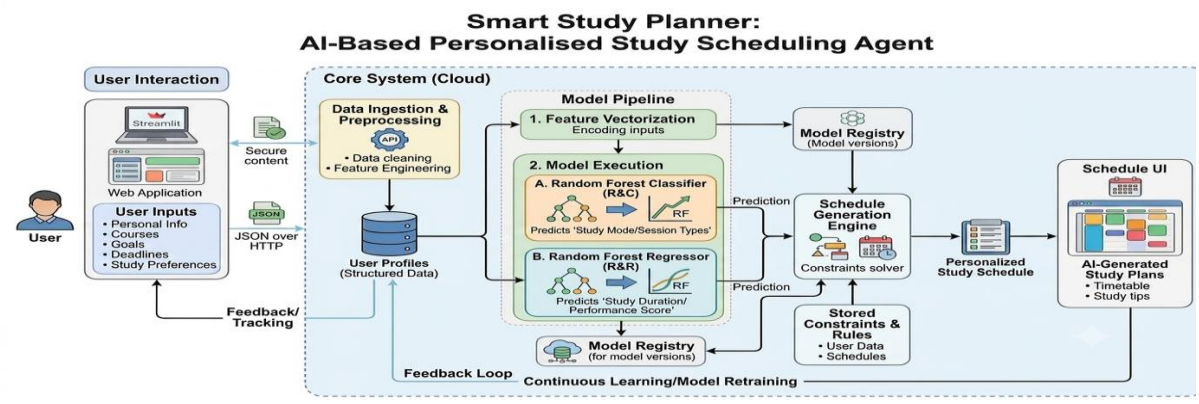
Existing study planning tools (MyStudyLife, Todoist, Notion, Anki) are passive — they record what the user enters but do not predict study time or automatically prioritize or reschedule. This project fills that gap by combining ML-based prediction, automatic prioritization, real-time visualization, and adaptive rescheduling in one application.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 Architecture Overview

The system follows a modular Streamlit architecture with four layers:

- Presentation Layer: Streamlit UI with custom CSS (injected via st.markdown).
- Application Logic Layer: main.py (routing/config), planner.py (form + ML), dashboard.py (charts/analytics).
- Machine Learning Layer: Trained scikit-learn models loaded via joblib, used for prediction.
- Data Layer: st.session_state stores subjects/schedule; scheduler.py builds the schedule DataFrame.



- ## 3.2 Module Structure

Module	File	Responsibility
App Entry Point	app/main.py	Config, CSS, navigation, routing
AI Planner	app/planner.py	Form, ML inference, My Subjects
Dashboard & Analytics	app/dashboard.py	Metrics, charts, analytics page
Scheduler	src/scheduler.py	Schedule generation, rescheduling
Utilities	src/utils.py	Formatting, validation, scoring
ML Training	src/ml_models.py	Model training and evaluation
Configuration	config.py	Subject list, paths, constants

3.3 Data Flow

1. Student fills the subject form (7 inputs) in AI Planner.
2. validate_subject_input() checks for logical errors.
3. compute_urgency_score() = remaining chapters / days until exam.
4. The 8-feature vector is sent to both ML models via predict_with_ml().
5. Regressor predicts study hours; Classifier predicts priority.
6. The enriched subject is stored in st.session_state.subjects.
7. generate_schedule() builds a Pandas DataFrame used by Dashboard and Analytics.

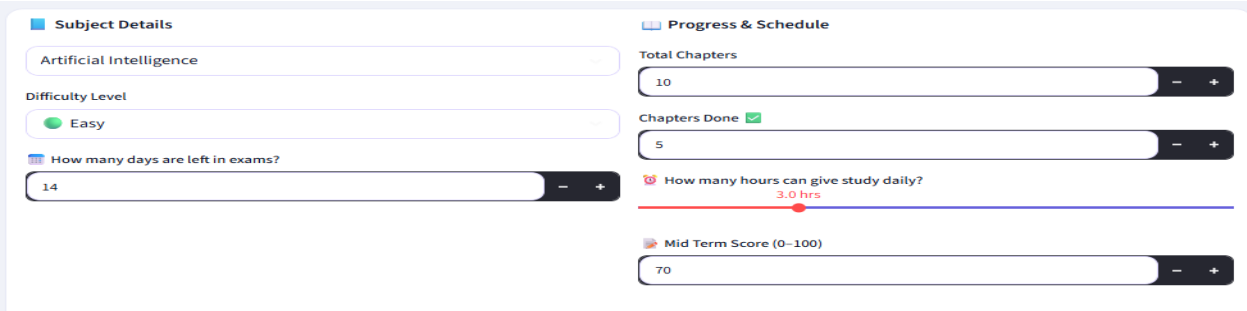
3.4 Technology Stack

Technology	Purpose
Python 3.10+	Primary programming language
Streamlit	Web app framework, UI, session state
scikit-learn	Random Forest Regressor and Classifier
Pandas	Schedule DataFrame and data manipulation
Plotly	Interactive bar, pie, and timetable charts
joblib	Saving/loading trained ML models

4. IMPLEMENTATION

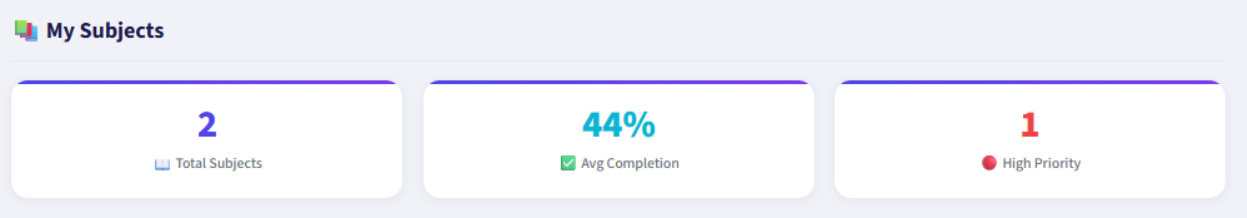
4.1 AI Planner Page

A two-column form collects: Subject Name, Difficulty (Easy/Medium/Hard), Days Until Exam, Total Chapters, Chapters Done, Daily Free Hours, and Mid Term Score. After validation, predict_with_ml() runs both models and shows a success message: "AI ne predict kiya: X.X hrs study time, Priority: LEVEL." The Generated Schedule section below then shows color-coded cards (red = high, yellow = medium, green = low) with exam date, daily hours, completion %, and total hours. A CSV export button downloads the full schedule, and a Clear All button resets everything.



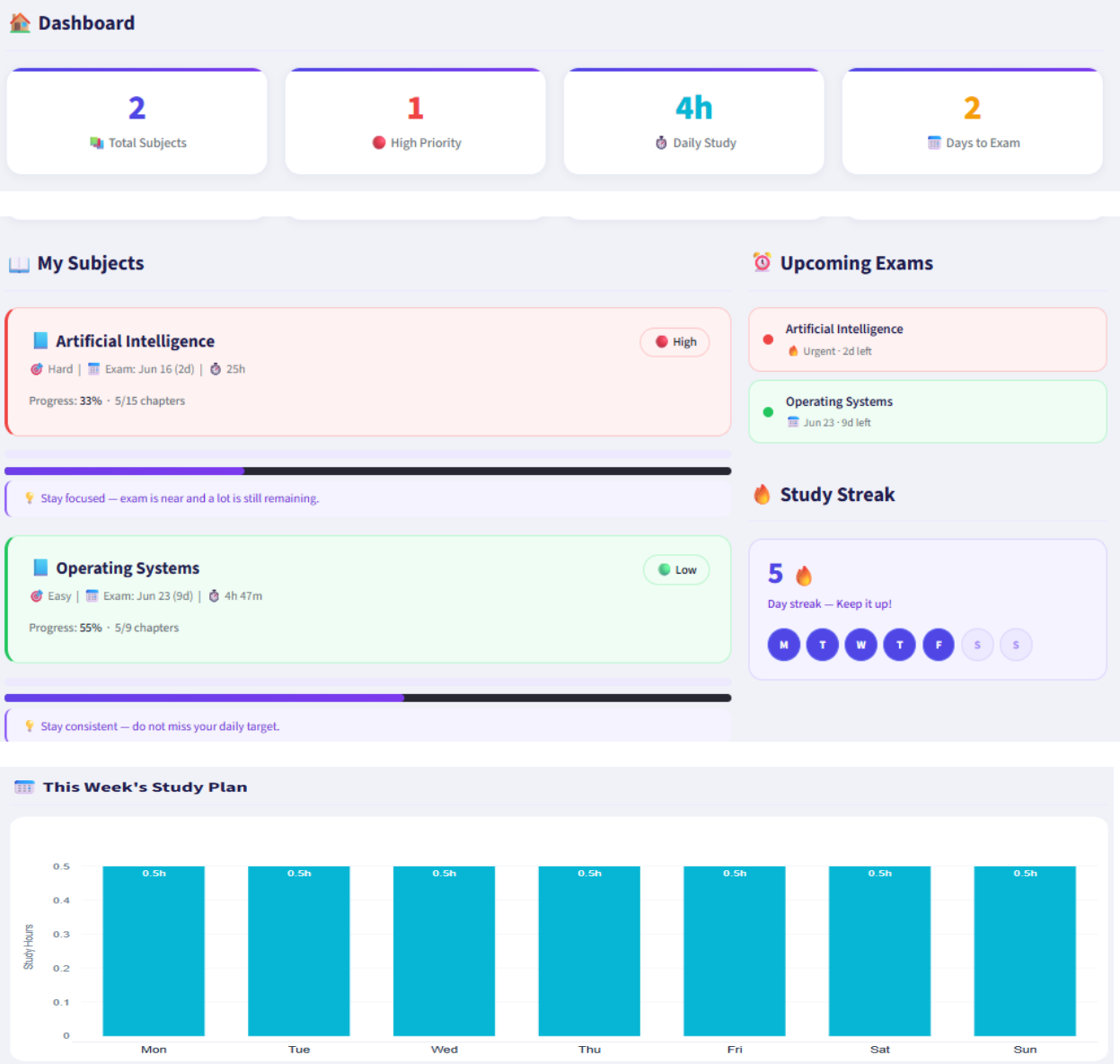
4.2 My Subjects Page

Shows 3 summary stats (total subjects, average completion %, high priority count) followed by a card per subject with progress bar, difficulty, exam date, and hours needed. Each card has a "Rescheduled" button (calls reschedule_missed() to upgrade priority when a session is missed) and a "Remove" button.



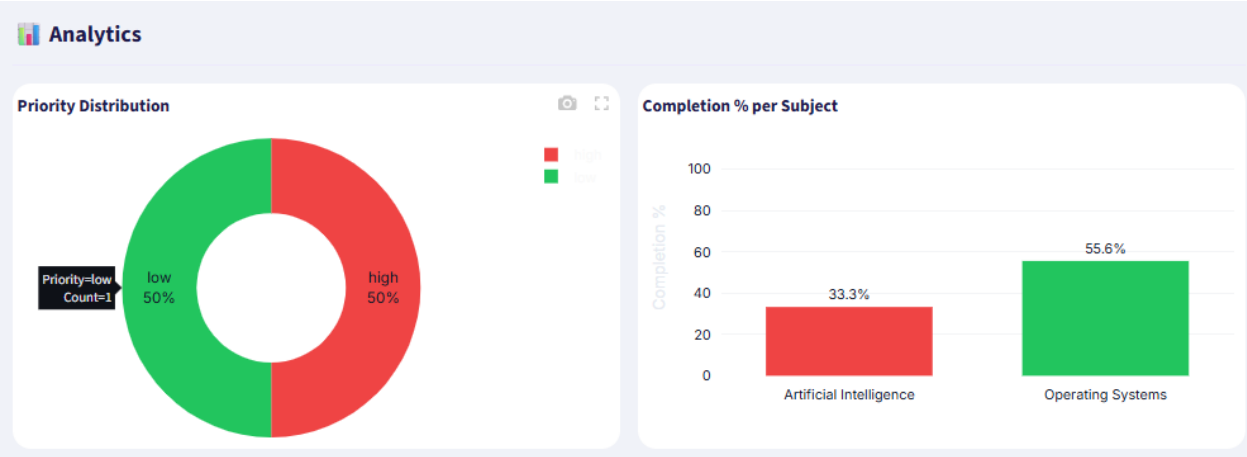
4.3 Dashboard Page

Shows four metric cards (Total Subjects, High Priority, Daily Study Hours, Days to Nearest Exam), a left column of subject progress cards with AI-generated study tips, a right column with Upcoming Exams (sorted by urgency) and a Study Streak tracker, and a weekly stacked bar chart (Mon-Sun) showing daily hours per subject.



4.4 Analytics Page

Shows a donut chart of priority distribution (High/Medium/Low) and a bar chart of completion % per subject, both color-coded by priority, followed by a full schedule summary table (Subject, Priority, Days Left, Done %, Daily Hrs, Total Hrs).



Schedule Summary

Subject	Priority	Days Left	Done %	Daily Hrs	Total Hrs
Artificial Intelligence	high	2	33.3	3	25
Operating Systems	low	9	55.6	1	4.8

5. MACHINE LEARNING METHODOLOGY

5.1 Problem Formulation

Two supervised learning tasks:

- Regression: predict total study hours needed (continuous, e.g., 8.5 hrs) from subject parameters.
- Classification: using the same parameters plus the predicted hours as a 9th feature, classify priority as Low (0), Medium (1), or High (2).

This two-stage design lets the Classifier use the workload estimate from the Regressor for a more informed priority decision.

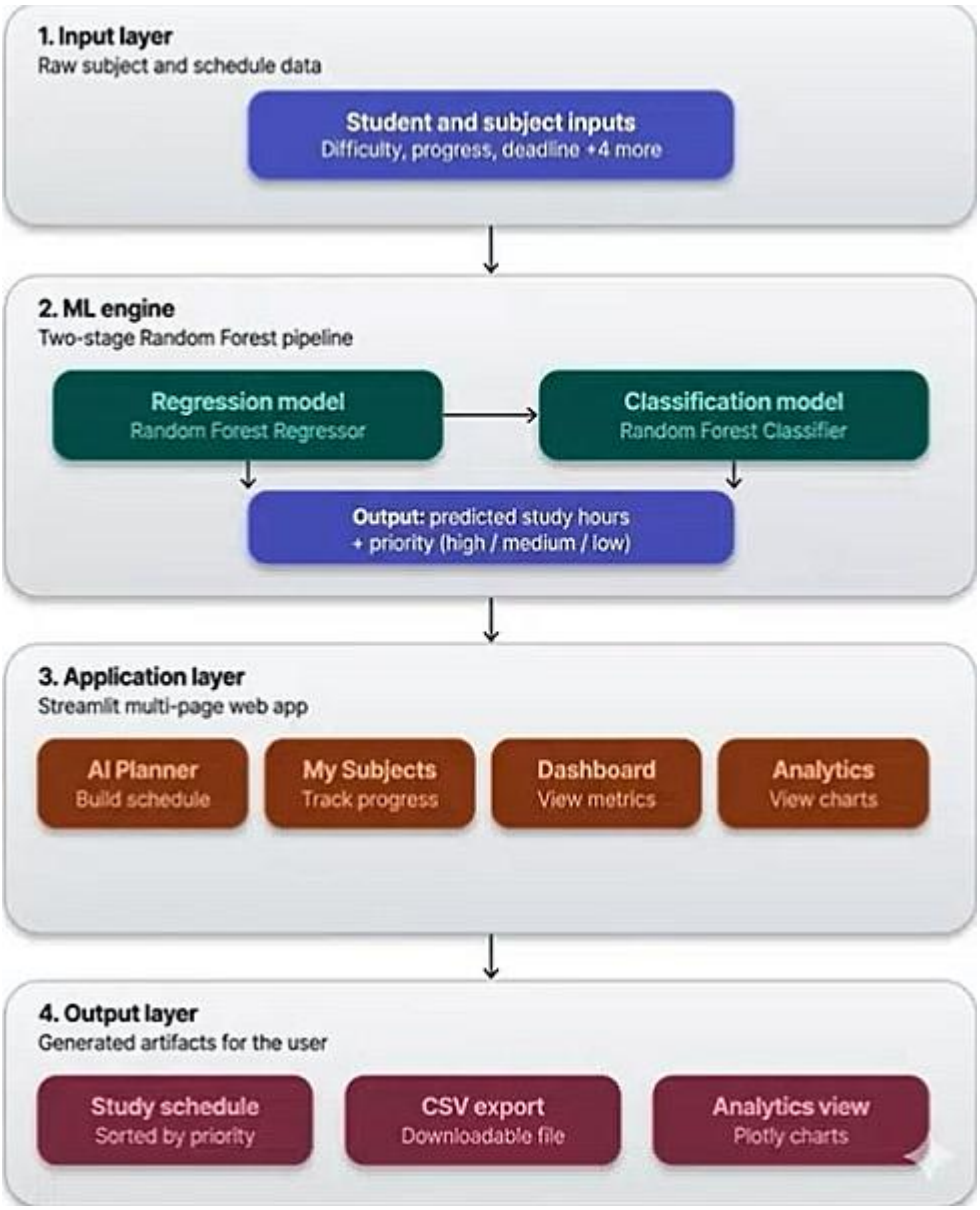
5.2 Feature Engineering

Feature	**Derivation**
difficulty_enc	easy=0, medium=1, hard=2
total_chapters	Direct input
remaining_chapters	total - chapters_done
completion_ratio	chapters_done / total_chapters
days_until_exam	Direct input

Feature	Derivation
daily_free_hours	Direct input
mid_score	Direct input
urgency_score	remaining_chapters / max(days_until_exam, 1)

5.3 Model Architecture

Both models are Random Forests, chosen for strong performance on tabular data, robustness to outliers, and resistance to overfitting. The Regressor takes the 8 features and outputs predicted study hours (clamped to a minimum of 0.5). The Classifier takes those 8 features plus the predicted hours (9 total) and outputs priority class 0/1/2, mapped to low/medium/high.


1. Input layer
Raw subject and schedule data
Student and subject inputs
Difficulty, progress, deadline +4 more
2. ML engine
Two-stage Random Forest pipeline
Regression model
Random Forest Regressor
Classification model
Random Forest Classifier
Output: predicted study hours
+ priority (high / medium / low)
3. Application layer
Streamlit multi-page web app
AI Planner
Build schedule
My Subjects
Track progress
Dashboard
View metrics
Analytics
View charts
4. Output layer
Generated artifacts for the user
Study schedule
Sorted by priority
CSV export
Downloadable file
Analytics view
Plotly charts

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

Smart Study Planner successfully combines Machine Learning with an interactive Streamlit application to deliver personalized, data-driven study schedules. All six project objectives were met: ML-based prediction and prioritization, real-time dashboard, analytics, adaptive rescheduling, and a professional UI. The two-stage ML pipeline (Regressor feeding the Classifier) produced more nuanced priority decisions than simple rule-based logic, and the modular code structure keeps each component independently testable.

6.2 Limitations

- Session state is not persistent — data is lost on refresh/close.
- Single-user only, no accounts/authentication.
- ML accuracy depends on training data size/quality.
- Weekly chart assumes equal hours every day.

6.3 Future Work

- Persistent database (SQLite/PostgreSQL) for multi-session storage.
- User authentication for multiple students.
- Mobile-responsive layout or dedicated app.
- Pomodoro timer and Google Calendar integration.
- Larger real-world dataset to retrain and improve ML models.
- Collaborative features (shared schedules, study groups).